SCHOOL OF COMPUTING AND ENGINEERING

University of West London

FINAL YEAR PROJECT REPORT

PhishKit: Phishing Awareness and Detection Toolkit

Abstract

Phishing remains a widespread cybersecurity challenge, cleverly exploiting human psychology to trick people into sharing sensitive credentials, financial details, and personal information. Even with many improvements in technical security measures, these attacks still succeed, mainly because they focus on human behaviour rather than on technical weaknesses. In this project, we explore how combining rule-based detection techniques with real-time user awareness strategies can work together to better prevent these deceptive attacks and keep everyone safer.

This report introduces the design, creation, and review of PhishKit, a comprehensive Phishing Awareness and Detection Toolkit. The system features a Python Flask backend with more than 24 heuristic detection rules spread across two analytical modules: a URL scanner that spots structural and behavioural signs of phishing, and an email analyser that uncovers social engineering tactics in sender domains, subject lines, and email content. A simple, self-contained HTML/CSS/JavaScript frontend offers an easy-to-use interface where users can paste datasets of URLs or email content for analysis. Every detection result comes with a helpful educational tip, guiding users to not only recognise that a threat is present but also understand exactly why it looks suspicious.

Testing with real-world phishing datasets showed that known phishing URLs consistently received high-to-critical risk scores (45–100), while legitimate URLs had scores of 0 or close to 0. The toolkit effectively identified all key threat types, such as brand impersonation, typo squatting, urgency tactics, and credential harvesting. The project concludes that transparent, explainable, rule-based detection, especially when combined with contextual user education, creates a more resilient defence than solely automated methods, which lack educational benefits for users.

Acknowledgements

I sincerely thank my project supervisor at the University of West London for their guidance, feedback, and support throughout this project. Their expertise in cybersecurity research greatly influenced the direction and depth of my work.

I also wish to acknowledge the academic and industry researchers whose published work on phishing detection, heuristic analysis, and user awareness training provided the theoretical foundation for this toolkit. Specifically, the contributions of Alkhalil et al. (2021), CrowdStrike (2024), and the Google Security Blog (2023) were instrumental in shaping the detection methodology.

Finally, I extend my sincere gratitude to the University of West London School of Computing and Engineering for providing the resources, tools, and academic environment that enabled the successful completion of this project.

Contents

Abstract **1**
Acknowledgements **3**
Code Extract Content **7**
Figure List **7**
Table list **8**
1. Introduction **9**
1.1 Aim and Objectives 9
1.1.1 Aim 9
1.1.2 Objectives 9
1.1.3 Project Motivation 10
2. Literature Review **11**
2.1 Overview of Phishing Attacks 11
2.2 Phishing Techniques and Indicators 11
2.3 Rule-Based Detection Methods 11
2.4 URL Rewriting and Evasion Techniques 12
2.5 User Awareness and Training 12
2.6 Summary and Research Gap 12
3. Research Methodology **14**
3.1 Literature Review and Foundation Building 14
3.2 Dataset Collection and Preparation 14
3.3 Rule Design and Development 14
3.4 System Development 14
3.5 Testing and Evaluation 15
4. Design and Implementation **16**
4.1 System Architecture 16
4.2 Scoring and Risk Classification Model 17
4.3 URL Detection Module 18
4.3.1 Module Structure 18
4.3.2 Threat Intelligence Constants 18
4.3.3 Detection Rules 18
4.3.4 Example Rule Implementation 19
4.3.5 Master Analysis Function 20
4.4 Email Detection Module 21
4.4.1 Module Structure 21
4.4.2 Example Rule — Sensitive Data Request 22
4.5 Flask REST API 23
4.5.1 Server Setup 23

4.5.2 API Endpoints 24
4.5.3 Endpoint Implementation 24
4.6 Frontend Interface 25
4.6.1 Design Decisions 25
4.6.2 Page Structure 25
4.6.3 URL Scanner Implementation 26
4.6.4 Email Analyser Implementation 28
4.6.5 Result Card Rendering 29
4.7 Awareness Page 30
4.8 Development Environment and Tools 32
5. Testing and Evaluation 33
5.1 Testing Approach 33
5.2 Datasets Used 34
5.2.1 URL Dataset 34
5.2.2 Email Dataset 34
5.3 Testing Script Structure 35
5.4 Test Results 36
5.4.1 URL Detection Results 37
5.4.2 Email Detection Results 37
5.4.3 Overall System Accuracy 37
5.5 Analysis of Results 38
5.5.1 URL Detection Performance 38
5.5.2 Email Detection Performance 38
5.5.3 Comparison of Modules 39
5.6 Limitations of the Testing 39
5.7 Testing Summary 40
Screenshot Summary 41
6. Discussion and Conclusion 41
6.1 Discussion 41
6.2 Limitations 42
6.3 Future Work 42
6.4 Conclusion 43
7. References 44
8. Appendix 45
8.1 Project File Structure 45
8.2 Setup Instructions 45
8.3 API Request Examples 45
8.4 Python Dependencies 46

Code Extract Content

Code Extract 1: DetectionFlag and AnalysisResult dataclass definitions 18
Code Extract 2: URL threat Intelligence constant lists 18
Code Extract 3: Brand Name in Subdomain rule 20
Code Extract 4: analyse_url master function 20
Code Extract 5: Sensitive Data Request rule 22
Code Extract 6: Flask server setup and CORS 24
Code Extract 7: check_url API endpoint 24
Code Extract 8: runURLScan JavaScript function 27
Code Extract 9 :runEmailScan JavaScript function 29
Code Extract 10: appendResult card rendering 30
Code Extract 11: Binary classification logic in testing.py 35
Code Extract 12: Project File Structure 45
Code Extract 13: UEL check API request example 45
Code Extract 14: Email check API request example 46

Figure List

Figure 1 - PhishKit project file structure as shown in Visual Studio Code Explorer. 17
Figure 2: url_detector.py showing the DetectionFlag dataclass and threat intelligence constants. 20
Figure 3: url_detector.py showing URL_SHORTENERS / SUSPICIOUS_TLDS constant lists 21
Figure 4: email_detector.py showing urgency and sensitive data keyword lists. 22
Figure 5: email_detector.py showing urgency and sensitive data keyword lists 23
Figure 6: email_detector.py showing urgency and sensitive data keyword lists 23
Figure 7: Flask development server starting successfully on port 5000. 25
Figure 8: PhishKit frontend showing the Checker page with the URL Scanner tab selected. 26
Figure 9: URLs to be scanned 27
Figure 10: URL Scanner results showing a safe URL scoring 0 and two phishing URLs scoring in the high to critical range. 28
Figure 11: Email Analyser tab showing a phishing email entered and a critical risk result returned. 29
Figure 12: Close-up of a phishing URL result card showing individual detection flags with severity colour coding. 30
Figure 13: Awareness page showing the six phishing technique cards and the pre-click safety checklist. 32
Figure 14: VS Code showing pip install output alongside the running Flask server. 33
Figure 15: testing.py open in VS Code showing the dataset loading and test execution functions. 36
Figure 16: Full terminal output from testing.py showing URL detection metrics, email detection metrics, and overall accuracy 36

Table list

Table 1: System Architecture Components 16
Table 2: Risk Score to Risk Level Mapping 17
Table 3: URL Detection Rules Summary 19
Table 4: Email Detection Rule Summary 22
Table 5: REST API Endpoints 24
Table 6: Frontend Page Structure 26
Table 7: Awareness Cards and Corresponding Detection Rules 31
Table 8: Development Tools and Technologies 33
Table 9: Binary Classification Metrics Definitions 34
Table 10: Datasets Used for Testing 35
Table 11: Testing SCript Functions 35
Table 12: URL Detection Overall Metrics 37
Table 13: URL Detection Precision, Recall and F1-Score by Class 37
Table 14: Email Detection Overall Metrics 37
Table 15: Email Detection Precision, Recall and F1-Score by Class 37
Table 16: Overall System Accuracy 37
Table 17: Comparison of URL and Email Detection Modules 39
Table 18: Screenshot Reference for Design and Implementation Chapter 41
Table 19: Python Package Dependencies 46

1. Introduction

The internet has truly changed the way we connect, work, and share our lives. At the same time, though, this digital world has opened new opportunities for cybercriminals to cause harm. One of the most common and serious threats is phishing—a sneaky type of social engineering that targets our trust and psychology to trick us into sharing private information, installing malicious software, or transferring money. Remember to stay vigilant and protect yourself online!

Phishing attacks have evolved significantly since they first appeared in the 1990s. What began as simple mass emails has become far more sophisticated, with targeted approaches such as spear phishing (targeting specific individuals), vishing (phone scams), smishing (SMS scams), and even social media-based tricks. Today's phishing methods — including URL rewriting, homograph domain attacks, and impersonating trusted brands — are designed to bypass security measures and evade automated detection (Alkhalil et al., 2021).

A key insight from research in this area is that phishing attacks often succeed more because they exploit human psychology than because of technical flaws in security systems. Common tactics like creating a sense of urgency ('Your account will be suspended in 24 hours'), pretending to be someone of authority ('HMRC Tax Refund Notice'), or offering rewards ('You have won a prize') tend to override rational doubt among users of all ages and technical skills. This important understanding forms the heart of this project's main message: that relying solely on technical detection is not enough, and that educating users should be an integral part of the detection approach.

This project introduces PhishKit, a friendly and effective Phishing Awareness and Detection Toolkit designed to meet both important needs. It combines a clear, rule-based detection system with real-time messages that help users stay aware of the context, making it easier to spot suspicious URLs and emails. Plus, it not only helps identify threats but also explains the techniques attackers use, empowering users with knowledge. The toolkit features a Python Flask REST API backend, two simple detection modules for URLs and emails, and a user-friendly HTML/CSS/JavaScript frontend that you can access easily without any installation required.

This report provides a comprehensive overview of our journey through the research, design, development, and evaluation process. It's been organised it to give smooth guidance: Chapter 1 sets out the aims and objectives along with the project motivation; Chapter 2 reviews the relevant literature on phishing attacks, detection methods and user awareness; Chapter 3 explains our research methodology; Chapter 4 details the design and implementation; Chapter 5 presents the results and analysis; and Chapter 6 concludes with discussion and concluding thoughts.

1.1 Aim and Objectives

1.1.1 Aim

To design, develop, and evaluate a rule-based phishing detection toolkit that identifies suspicious URLs and emails while simultaneously providing real-time educational feedback to users, thereby integrating technical threat detection with user awareness training within a single comprehensive system.

1.1.2 Objectives

1. Analyse different types of phishing attacks, including spear phishing, clone phishing, and business email compromise.

2. Study attacker techniques such as URL manipulation, email spoofing, and social engineering cues.
3. Research rule-based detection approaches, including heuristics, regex, and blocklists.
4. Explore user psychology and awareness training methods.
5. Develop a prototype detection toolkit integrating rule-based alerts and awareness messages.
6. Test and refine rules and awareness strategies using real datasets.

1.1.3 Project Motivation

This project is driven by the increasing complexity of phishing attacks and the persistent vulnerability among users. Although machine learning detection systems are effective, they can be difficult and require large, high-quality databases. Conversely, rule-based systems are more transparent, easier to understand, and simpler to implement, which makes them ideal for educational purposes.

By combining rule-based detection with awareness messages, this project aims to empower users to identify phishing attempts independently. This accessible approach provides immediate protection and promotes lasting behavioural change, contributing to safer digital environments for all.

2. Literature Review

2.1 Overview of Phishing Attacks

Phishing is a type of cybercrime in which attackers impersonate trustworthy entities via digital messages to trick people into revealing sensitive information or taking actions that aren't in their best interests (Alkhalil et al., 2021). It all started back in 1996, when hackers aimed at AOL users, and the unique 'ph' spelling comes from early hacker culture. Over the years, phishing has grown to be the most common way cybercriminals gain initial access, with billions of phishing emails sent around the world every day.

Alkhalil et al. (2021) offer a valuable and detailed classification of different phishing attack types in their significant study. They highlight spear phishing — very targeted attacks against specific individuals using personalised information — as the most hazardous type, leading to the most successful business email compromise (BEC) cases. Clone phishing, where a legitimate email is copied with a malicious link added, is especially tricky to spot because the email closely resembles trusted communications the user has already received and trusted.

Vishing (voice phishing) and smishing (SMS phishing) now go beyond email, expanding the ways attackers can target us. Social media phishing, where scammers create convincing fake profiles or hack into real accounts, has become a big concern, especially for users on LinkedIn and Instagram (CrowdStrike, 2024). With so many ways to attack, relying on just one defence isn’t enough to keep us safe.

2.2 Phishing Techniques and Indicators

Lenaerts-Bergmans (2024) at CrowdStrike thoughtfully identifies the main technical signs of phishing attacks that help us spot them. These include URL tricks like typosquatting (registering domains with intentional misspellings, such as 'arnazon.com'), homograph attacks (using Unicode characters that look very similar to normal ASCII ones), and subdomain exploitation (where attackers embed familiar brand names as subdomains within their own domains, like 'paypal.com.evil-site.xyz').

Email spoofing techniques include display name deception, where attackers might show 'PayPal Security' as the sender even though the actual address is 'noreply@free-mail-service.com'. They also use domain lookalike registration, such as 'paypa1.com' with a digit substituted for a letter and take advantage of free email services to make messages appear more trustworthy. Although header forgery has become less common due to the widespread adoption of authentication standards such as SPF, DKIM, and DMARC (DMARC.org, 2017), some attacks still attempt to bypass email security on systems lacking adequate protections.

Social engineering manipulation is thoroughly explored by John (2025) in a ResearchGate study focused on user psychology. The research highlights six key psychological triggers that hackers often exploit in phishing attacks: urgency (pressuring individuals to act quickly without thinking), authority (pretending to be trusted institutions like HMRC, employers, or banks), social proof (falsely claiming others have already complied), scarcity (creating a false sense of limited availability), fear (raising concerns about negative consequences), and reciprocity (offering something valuable in exchange for personal details).

2.3 Rule-Based Detection Methods

Rule-based detection systems are among the most mature and widely deployed approaches to phishing identification. Unlike machine learning models, rule-based systems rely on explicitly defined logical conditions — heuristics, regular expressions, and keyword matching — making their decision-making fully transparent and auditable. Liu et al. (2021) note that this

transparency is a significant practical advantage in security contexts, where the ability to explain and justify detections to non-technical users is as important as detection accuracy.

According to the Google Security Blog (2023), rule-based detection remains the primary method for large-scale phishing filtering at Google. It highlights that clear indicators such as mismatched sender domains, suspicious URL structures, and unusual header formats are effective against most phishing attacks, which often reuse tested templates rather than inventing new ones. The study acknowledges that although advanced zero-day phishing attacks might bypass rule-based defences, most real-world attacks can still be identified through pattern recognition.

Heuristic URL analysis is a recognised subfield of rule-based detection. Key indicators noted in the literature include the use of IP addresses instead of domain names (Alkhalil et al., 2021), very long URLs that hide the true domain, URL shortening services that conceal the destination, the @ symbol in URLs which browsers interpret as credentials, deep subdomain nesting, and high-risk top-level domains like .xyz, .tk, and .ml, which are often linked to malicious registrations (Systems-x, 2022).

2.4 URL Rewriting and Evasion Techniques

A recent significant development is the abuse of URL rewriting services by malicious actors for phishing attacks. French (2024), in an SC Media report, highlights a sharp rise in cases in which cybercriminals exploit the URL-rewriting capabilities of legitimate Secure Email Gateways (SEGs). They craft convincing redirect links that conceal malicious websites. One well-known technique, called 'double rewriting,' involves attackers applying two layers of rewriting: initially wrapping a malicious URL with Proofpoint's service, then wrapping it again with INKY. This creates a link that can bypass whitelist filters because it appears to originate from a trusted SEG provider.

This evasion technique is important for rule-based detection because it shows that relying on a single indicator—such as a reputable domain prefix—is insufficient. Detection systems need to analyse multiple independent heuristics simultaneously to accurately assess threats.

2.5 User Awareness and Training

John (2025) on ResearchGate emphasises that user awareness training is an essential supplement to technical detection methods. Their study indicates that users who receive contextual, real-time explanations of why a communication is suspicious are more likely to remember and apply this knowledge in future phishing attempts than those who receive only simple 'safe/unsafe' labels from automated systems. This finding supports PhishKit's decision to include educational awareness tips with each detection alert.

The Cyber Sierra security knowledge base (Knowledge, 2025) examines the 'psychology of phishing' and provides empirical evidence that even highly skilled users are susceptible to attacks that leverage emotional triggers such as time pressure and impersonation of authority figures. This supports Cyber Security News (2025), which claims that no matter how advanced the technical defences are, they cannot replace user training. Consequently, a combined approach that includes both technical measures and user education is crucial for robust security.

2.6 Summary and Research Gap

The literature review clearly agrees that phishing attacks succeed by exploiting human psychology. It also notes that rule-based detection systems are effective and transparent for identifying known attack patterns. Additionally, user education remains a vital but often separate part of anti-phishing efforts. The research gap this project aims to fill is the lack of

practical, integrated tools that merge transparent heuristic detection with real-time, contextual user education in a single, accessible interface. Current tools generally provide detection results without explanations, missing the chance to strengthen user resilience alongside threat detection. PhishKit is developed specifically to fill this gap.

3. Research Methodology

This project used a Design Science Research (DSR) approach, suitable for creating a functional artefact, specifically, a working software toolkit. DSR consists of repeated cycles of designing, building, and evaluating, with each iteration improving the artefact based on performance observations. The research was organised into seven sequential phases that follow the methodology described in the project proposal.

3.1 Literature Review and Foundation Building

The initial phase consisted of a systematic review of academic and industry sources on phishing attack methods, heuristic detection, and user awareness. Sources included peer-reviewed journals (via Google Scholar, ACM Digital Library, and IEEE Xplore), well-regarded industry publications (CrowdStrike, Google Security Blog, SC Media), and academic theses. The review focused on three main research questions:

- How can rule-based detection methods, combined with user awareness strategies, effectively reduce the success of phishing attacks?
- How effective are heuristic-based phishing indicators compared to modern evasion techniques used by attackers?
- What impact does real-time user feedback have on reducing risky click behaviour during phishing attempts?

The literature review established the theoretical basis for the detection rule sets and informed the design of the awareness messaging component.

3.2 Dataset Collection and Preparation

Phishing and legitimate URL datasets were collected from Phishtank (phishtank.org), a community-driven repository of verified phishing URLs, and from Kaggle phishing datasets. The datasets were reviewed, cleaned, and categorised by attack type. Legitimate baseline URLs were sourced from the Alexa Top 1000 most-visited domains list to ensure representativeness. The datasets were used during the testing and evaluation phases rather than for training, consistent with the project's rule-based (non-machine-learning) approach.

3.3 Rule Design and Development

Detection rules were created based on patterns found in the literature review and validated with collected datasets. Each rule includes a descriptive name, the targeted phishing pattern, supporting literature evidence, a severity level (low/medium/high/critical), a score impact, and an awareness tip for non-technical users explaining the rule. These rules were repeatedly tested and adjusted to reduce false positives while ensuring they still detect actual phishing attempts.

3.4 System Development

The system was built using a full-stack approach. The backend, developed in Python 3 with Flask, includes detection logic for URL and email analysis in separate modules. The frontend consists of a single HTML file with embedded CSS and JavaScript, requiring no build tools or external dependencies. Development took place in Visual Studio Code with Git for version control.

The development process used an iterative method: each detection rule was individually implemented and tested before being combined into the complete module. API endpoints were

checked with manual HTTP requests prior to frontend integration. Finally, the entire system was tested comprehensively with both phishing and legitimate samples from the datasets.

3.5 Testing and Evaluation

System evaluation was carried out based on three main criteria: detection accuracy (correctly classifying known phishing samples as high or critical risk), false positive rate (misidentifying legitimate URLs or emails), and clarity of the awareness messages (evaluated by comparing the educational tips with recognised plain-language writing standards). Additionally, a qualitative review of the awareness tips was performed by comparing them to the psychological manipulation indicators outlined in the literature review.

4. Design and Implementation

4.1 System Architecture

PhishKit operates on a client-server architecture, distinctly separating the detection engine from the presentation layer. It consists of three main parts: the Flask REST API server, the detection modules library, and the browser-based frontend client.

Component	Role and Technology
app.py	Flask REST API server — receives requests, delegates to detectors, returns JSON
url_detector.py	URL analysis engine — 14 heuristic rules, scoring model, risk classification
email_detector.py	Email analysis engine — 10 heuristic rules targeting social engineering patterns
index.html	Self-contained frontend — HTML, CSS and JavaScript in one file, runs in any browser
Component	Role and Technology
app.py	Flask REST API server — receives requests, delegates to detectors, returns JSON

Table 1: System Architecture Components

The frontend communicates with the backend only through two POST endpoints. No database is used; all analysis is performed in memory for each request. This stateless design keeps the system lightweight and simple to run on any machine with Python installed.

Here's the data flow: the user enters input in the browser, prompting JavaScript to send a POST request to the Flask server. The server directs this input to the relevant detector module, which processes all rules and generates a scored output. Flask then serialises this result into JSON, and JavaScript displays the result cards clearly on the screen.

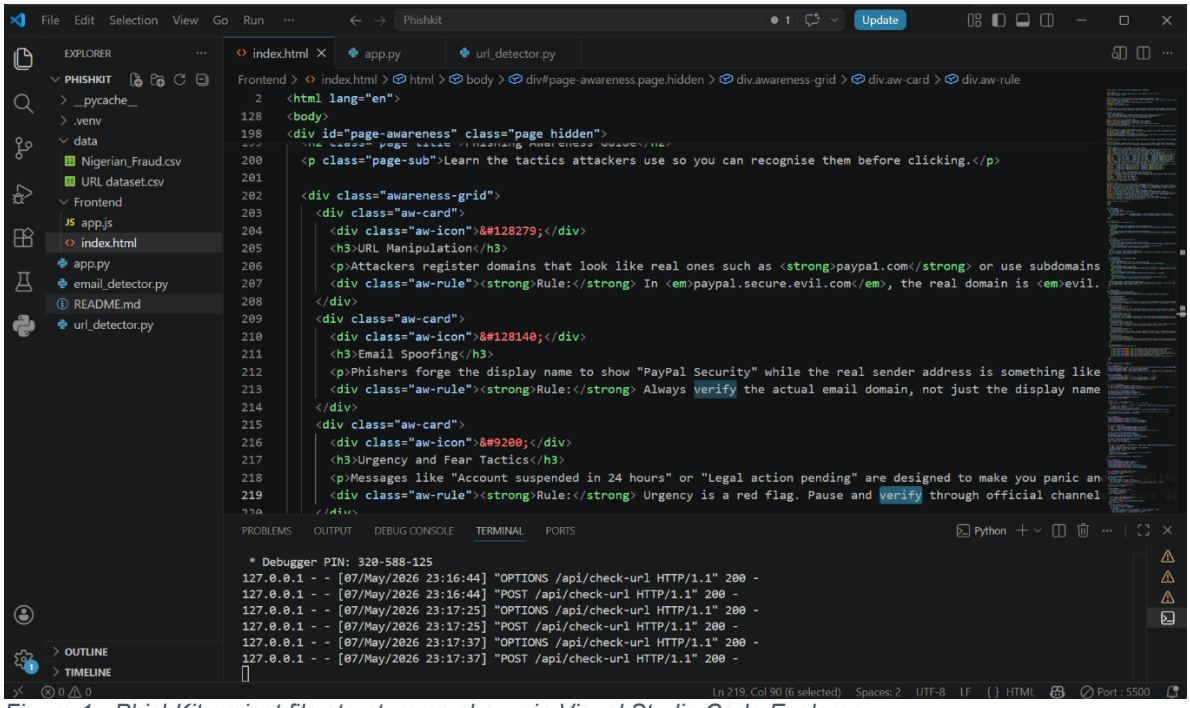


Figure 1 - PhishKit project file structure as shown in Visual Studio Code Explorer.

4.2 Scoring and Risk Classification Model

Both detection modules use a consistent additive scoring system, where each activated rule increments the risk score by a set number of points. The overall score is capped at 100 and maps to a risk level as described below.

Score Range	Risk Level	Meaning
0	Safe	No suspicious indicators detected
1–19	Low	Minor flags present; exercise normal caution
20–44	Medium	Multiple risk factors identified; verify before acting
45–69	High	Strong phishing indicators; likely malicious
70–100	Critical	Multiple high-severity indicators; do not visit/act

Table 2: Risk Score to Risk Level Mapping

The score cap prevents scores from rising too high when many low-severity rules trigger simultaneously. Each rule's severity weight reflects the strength of its evidence: rules like IP Address as Host and @ Symbol in URL have high weights due to their rarity in legitimate cases, while rules like Phishing Keyword Detected have lower weights because their keywords can also appear in authentic URLs.

4.3 URL Detection Module

4.3.1 Module Structure

The URL detection module resides in url_detector.py. It includes two data classes, a collection of threat-intelligence constants, 14 rule functions, and a main analysis function. The data class structure guarantees that all results are organised and uniform, regardless of which rules are activated.

```
# Dataclasses define the structure of every result
@dataclass
class DetectionFlag:
    rule_name: str       # name shown in the frontend
    description: str     # what was specifically found
    severity: str        # low / medium / high / critical
    score_impact: int    # points added to total score
    awareness_tip: str   # educational message for the user

@dataclass
class AnalysisResult:
    url: str
    score: int           # 0-100 total risk score
    risk_level: str      # safe / low / medium / high / critical
    flags: List[DetectionFlag]
    summary: str
```

Code Extract 1: DetectionFlag and AnalysisResult dataclass definitions

4.3.2 Threat Intelligence Constants

Rather than hardcoding values inside each rule function, shared threat data is defined as module-level constants. This makes the lists easy to extend without modifying rule logic.

```
URL_SHORTENERS   = {"bit.ly", "tinyurl.com", "t.co", "ow.ly", ...}
SUSPICIOUS_TLDS  = {".xyz", ".top", ".tk", ".ml", ".ga", ...}
BRAND_KEYWORDS   = {"paypal", "amazon", "microsoft", "apple", ...}
PHISHING_KEYWORDS= {"login","verify","secure","suspend","confirm",...}
LEGIT_DOMAINS    = {"google.com", "paypal.com", "amazon.com", ...}
```

Code Extract 2: URL threat Intelligence constant lists

4.3.3 Detection Rules

Each rule is a standalone function that accepts the raw URL string and a parsed URL object and returns a list of DetectionFlag instances. The table below summarises all fourteen rules.

Rule Name	Severity	Score	What It Detects
IP Address as Host	High	+30	Raw IPv4 address used instead of a domain name

Excessively Long URL	Medium	+15	URL exceeds 150 characters — padding to obscure domain
URL Shortener Detected	Medium	+20	Domain matches known shortening service — hides destination
@ Symbol in URL	High	+35	@ present — browser treats pre-@ content as credentials
Excessive Subdomains	High	+25	Five or more subdomain levels — brand impersonation pattern
Brand Name in Subdomain	Critical	+40	Known brand appears in subdomain of a non-brand domain
High-Risk TLD	Medium	+15	Domain uses a TLD with high phishing prevalence
Non-HTTPS Connection	Medium	+15	Plain HTTP — no encryption or server verification
Multiple Phishing Keywords	High	+25	Three or more phishing-associated terms found in URL
Phishing Keyword Detected	Low	+8	One or two phishing-associated terms found in URL
Multiple Hyphens in Domain	Medium	+15	Three or more hyphens in domain name
Heavy URL Encoding	Medium	+18	Four or more percent-encoded characters — obfuscation
Open Redirect Parameter	Medium	+18	Query string contains redirect or return parameters
Possible Typosquatting	High	+35	Domain closely resembles a legitimate brand domain

Table 3: URL Detection Rules Summary

4.3.4 Example Rule Implementation

The provided extract illustrates the Brand Name in Subdomain rule, which is the top-scoring URL rule. It verifies whether a recognised brand name appears in the subdomain portion of a URL, even when the main domain does not belong to that brand.

```
def _check_brand_in_subdomain(url, parsed):
    host  = parsed.netloc.lower().replace('www.', '')
    parts = host.split('.')
    if len(parts) < 3:
        return []
    subdomain     = '.'.join(parts[:-2])   # everything before final two parts
    actual_domain = '.'.join(parts[-2:])   # the real domain
    for brand in BRAND_KEYWORDS:
        if brand in subdomain and brand not in actual_domain:
            return [DetectionFlag(
                rule_name    = 'Brand Name in Subdomain',
                description  = f"'{brand}' appears in subdomain but real domain is '{actual_domain}'",
                severity     = 'critical',
```

```
            score_impact = 40,
            awareness_tip= f'The real {brand} website is {brand}.com ...'
        )]
    return []
```

Code Extract 3: Brand Name in Subdomain rule

4.3.5 Master Analysis Function

The analyse_url function acts as the entry point. It normalises the URL, runs every rule, totals the scores, and returns a complete AnalysisResult.

```
ALL_RULES = [
    _check_ip_address, _check_url_length, _check_url_shortener,
    _check_at_symbol,  _check_multiple_subdomains, ...
]

def analyse_url(raw_url):
    parsed    = urllib.parse.urlparse(url)
    all_flags = []
    for rule_fn in ALL_RULES:
        all_flags.extend(rule_fn(url, parsed))   # each rule adds 0 or more flags
    total_score = min(100, sum(f.score_impact for f in all_flags))
    risk        = _score_to_risk(total_score)    # maps number to label
    return AnalysisResult(url, total_score, risk, all_flags, summary)
```

Code Extract 4: analyse_url master function

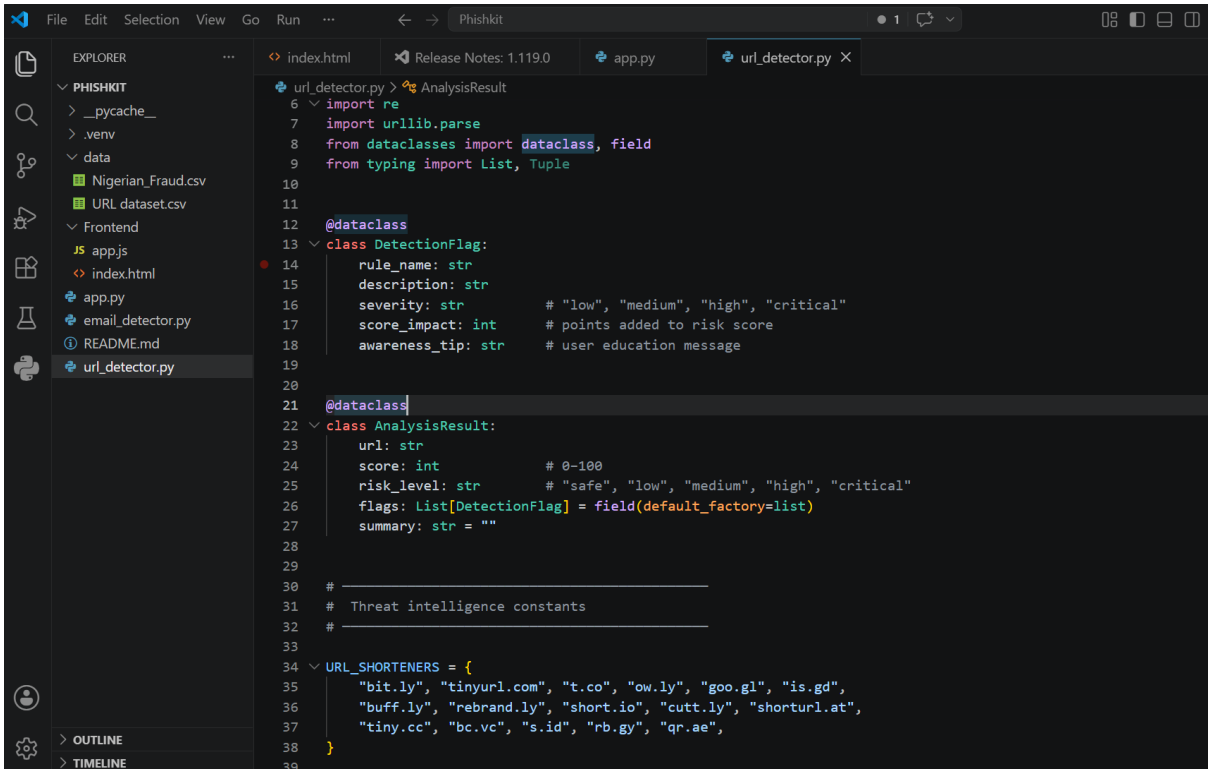


Figure 2: url_detector.py showing the DetectionFlag dataclass and threat intelligence constants.

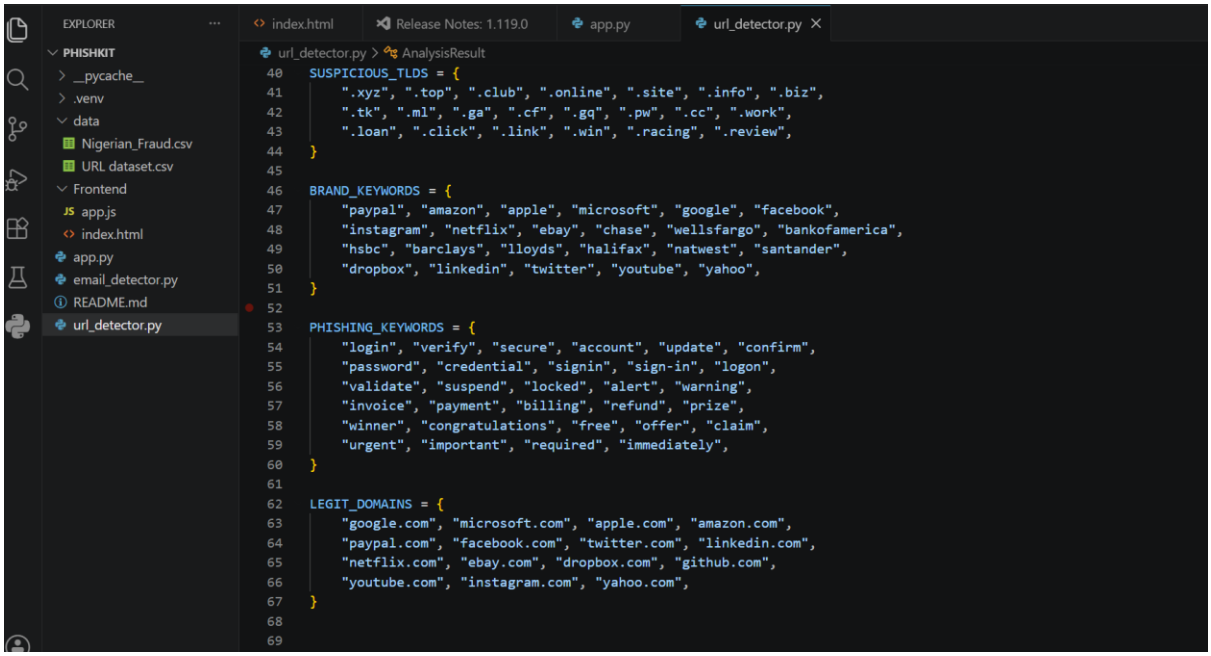


Figure 3: url_detector.py showing URL_SHORTENERS / SUSPICIOUS_TLDS constant lists

4.4 Email Detection Module

4.4.1 Module Structure

The email detection module is implemented in email_detector.py and follows the same structural pattern as the URL module. It analyses three string inputs — sender, subject, and body — and applies 10 rule functions to target social engineering patterns.

Rule Name	Severity	Score	What It Detects
High Urgency Language	High	+25	Three or more urgency trigger words such as urgent, expire, immediately
Urgency Language Detected	Medium	+12	One or two urgency trigger words present
Threatening Language	High	+28	Phrases such as legal action, account closed, or law enforcement
Too-Good-To-Be-True Offer	High	+30	Prize, lottery, winner, or inheritance language detected
Sensitive Data Request	Critical	+45	Explicit request for passwords, card numbers, PINs, or government IDs
Brand Impersonation via Free Email	Critical	+50	Claims to be a brand, but the sender uses Gmail, Yahoo, or similar
Suspicious Sender Domain TLD	High	+25	Sender domain uses a high-risk TLD such as .xyz or .top
Spoofed Sender Domain	Critical	+45	Sender domain resembles but is not an actual brand domain
Generic Greeting	Low	+8	Non-personalised greeting, such as Dear Customer or Dear User

Dangerous Attachment Type	Critical	+40	Email body references executable file extensions such as .exe or .vbs
Phishing Language Pattern	Medium	+15	Advance-fee fraud phrases such as kindly revert or do the needful

Table 4: Email Detection Rule Summary

4.4.2 Example Rule — Sensitive Data Request

The Sensitive Data Request rule has the highest single-point impact of any email rule at +45 points, reflecting that no legitimate organisation ever requests passwords, card numbers, or government identifiers via email.

```
SENSITIVE_DATA_REQUESTS = [
    'enter your password', 'confirm your password',
    'credit card number', 'cvv', 'pin number',
    'social security', 'national insurance', ...
]

def _check_sensitive_data_request(subject, body, sender):
    combined = (subject + ' ' + body).lower()
    matched  = [w for w in SENSITIVE_DATA_REQUESTS if w in combined]
    if matched:
        return [DetectionFlag(
            rule_name    = 'Sensitive Data Request',
            severity     = 'critical',
            score_impact = 45,
            awareness_tip= 'No legitimate organisation will ask for your
                            password or card number via email ...'
        )]
    return []
```

Code Extract 5: Sensitive Data Request rule

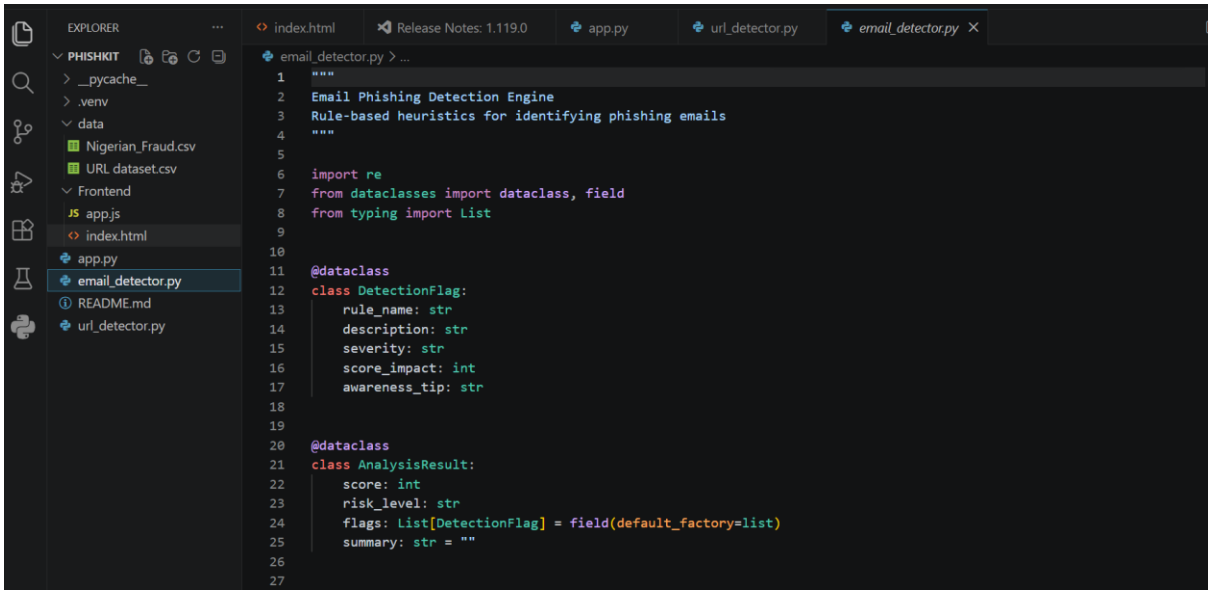


Figure 4: email_detector.py showing urgency and sensitive data keyword lists.

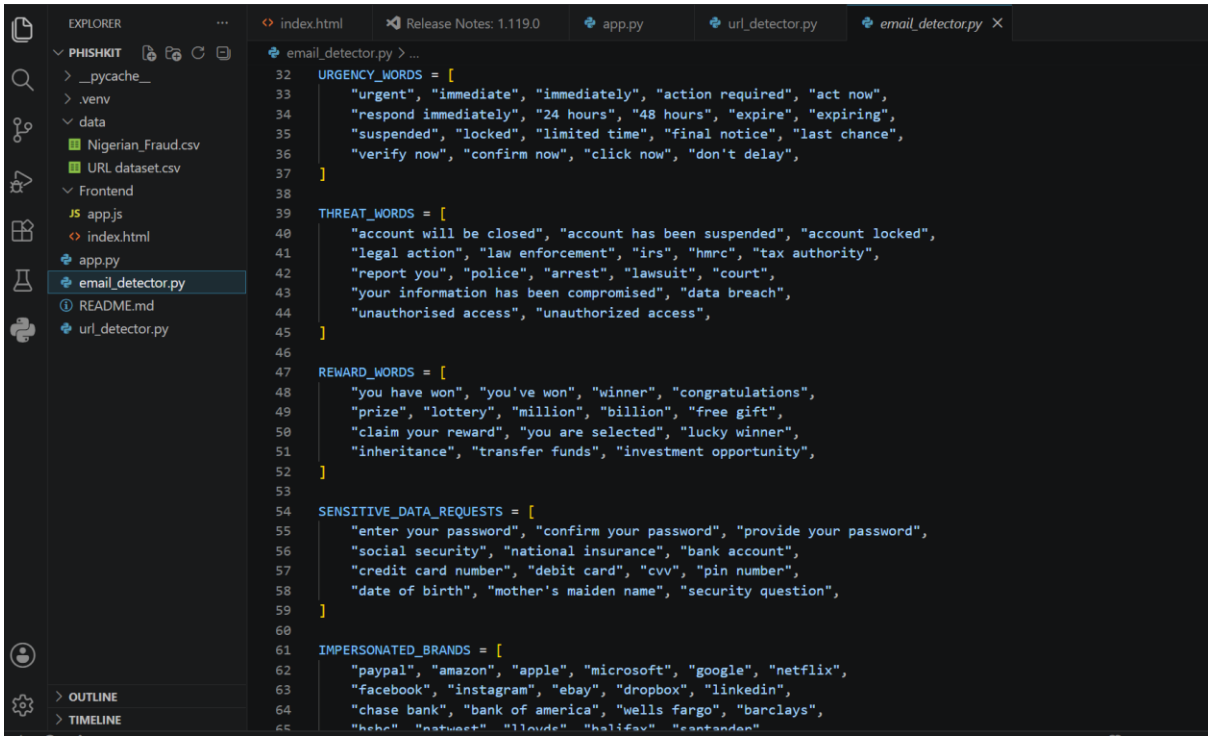


Figure 5: email_detector.py showing urgency and sensitive data keyword lists

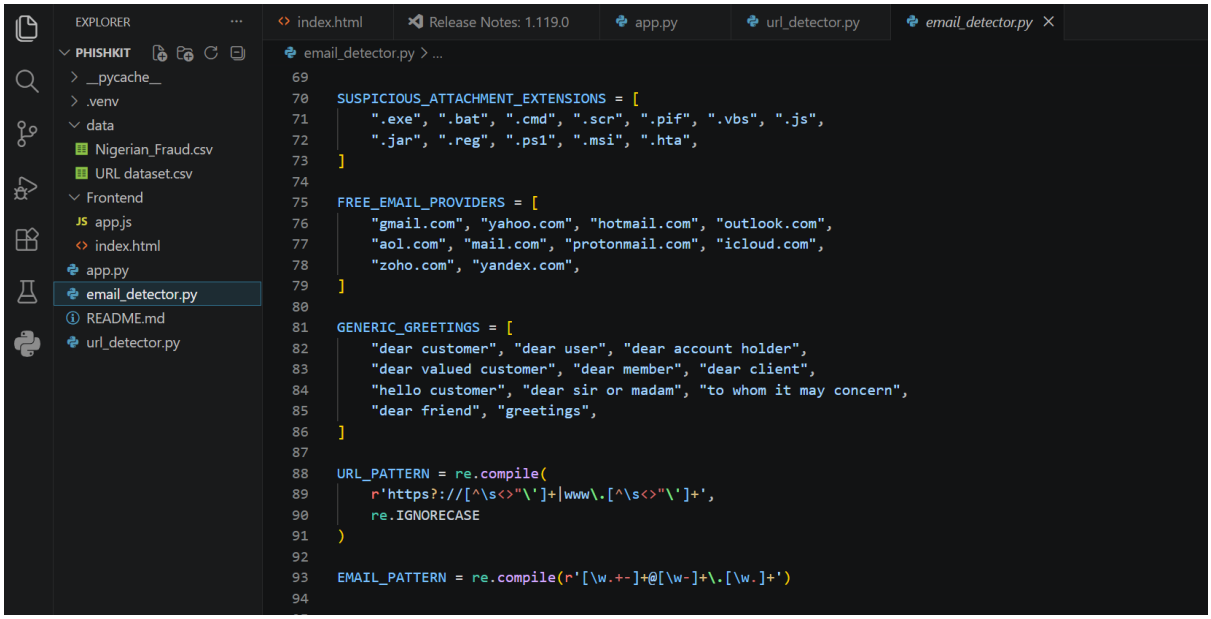


Figure 6: email_detector.py showing urgency and sensitive data keyword lists

4.5 Flask REST API

4.5.1 Server Setup

The Flask server is implemented in app.py. It imports the two analysis functions from the detectors package, configures CORS to allow cross-origin browser requests, and exposes three HTTP endpoints. The server acts purely as a routing and serialisation layer — all detection logic lives in the detector modules.

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from detectors import analyse_url, analyse_email

app = Flask(__name__)
CORS(app)   # allows the browser frontend to call the API
```

Code Extract 6: Flask server setup and CORS

4.5.2 API Endpoints

Endpoint	Method	Input	Output
/api/health	GET	None	{ status: ok }
/api/check-url	POST	{ url: string }	{ score, risk_level, summary, flags[] }
/api/check-email	POST	{ subject, body, sender }	{ score, risk_level, summary, flags[] }

Table 5: REST API Endpoints

4.5.3 Endpoint Implementation

Each POST endpoint consistently follows this pattern: validate input, invoke the appropriate detector, serialise the resulting data class into a dictionary, and return the data as JSON.

```
@app.route('/api/check-url', methods=['POST'])
def check_url():
    data    = request.get_json()
    raw_url = str(data['url']).strip()
    result  = analyse_url(raw_url)       # calls url_detector.py
    payload = {
        'url':        result.url,
        'score':      result.score,
        'risk_level': result.risk_level,
        'summary':    result.summary,
        'flags': [{
            'rule_name':    f.rule_name,
            'description':  f.description,
            'severity':     f.severity,
            'score_impact': f.score_impact,
            'awareness_tip':f.awareness_tip
        } for f in result.flags]
    }
    return jsonify(payload), 200
```

Code Extract 7: check_url API endpoint

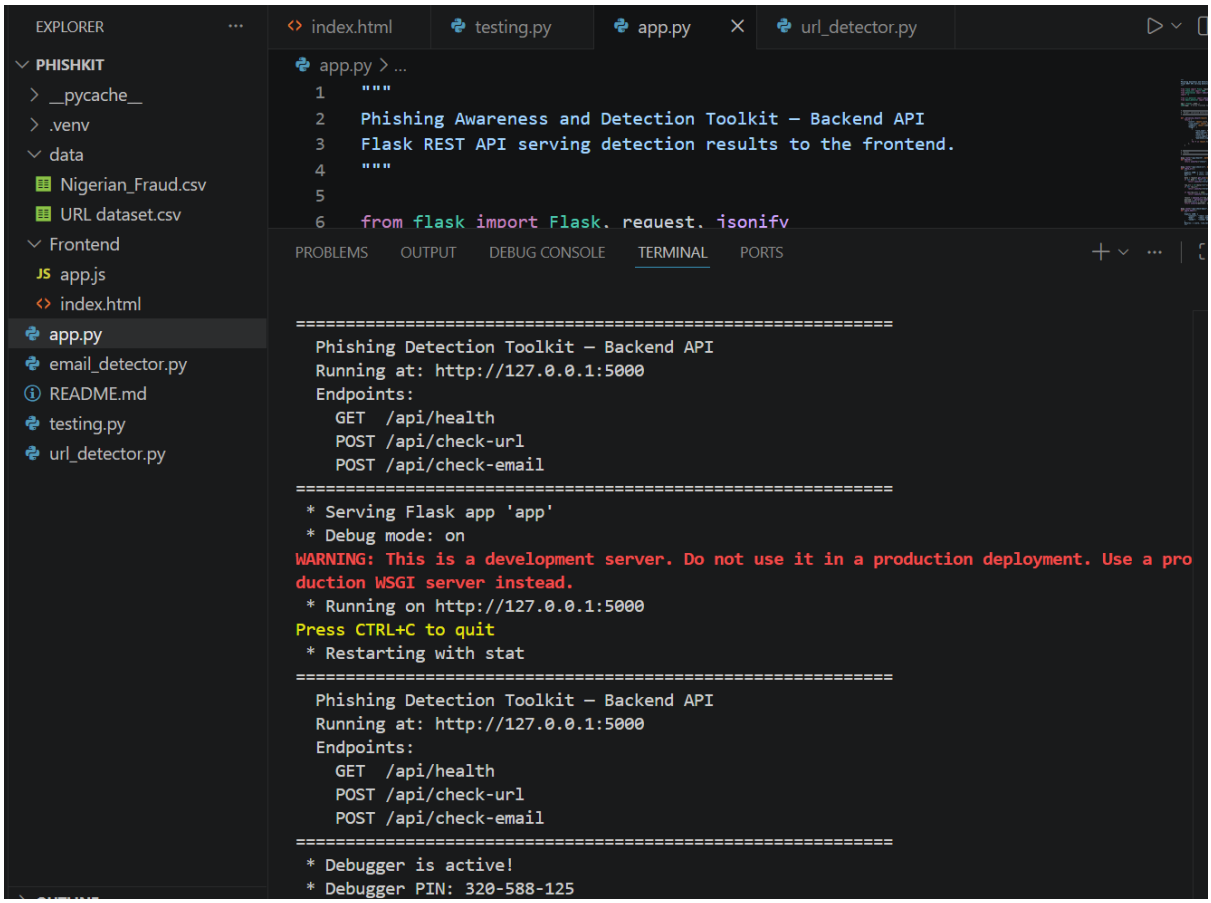


Figure 7: Flask development server starting successfully on port 5000.

4.6 Frontend Interface

4.6.1 Design Decisions

The frontend was built on three main principles. Firstly, it had to be self-contained, a single HTML file with all CSS and JavaScript embedded, allowing it to run just by double-clicking, without any build process or web server needed. Secondly, it needed to be straightforward, avoiding unnecessary features, forms, or navigation. Thirdly, each result had to instantly display the relevant information without any additional clicks to expand.

The colour scheme features a dark background with cyan accents for interactive elements. Risk levels are consistently colour-coded: cyan for safe, orange for medium, and red for high and critical, providing users with an immediate visual cue about severity before reading any text.

4.6.2 Page Structure

The interface features two pages accessible through the navigation header. When a navigation button is clicked, it triggers the showPage() JavaScript function, which hides the current page and shows the selected page without reloading.

Page	Contents
Checker	Two tabs: URL Scanner for pasting and scanning multiple URLs, Email Analyser for entering sender, subject, and body content.
Awareness	Six educational cards explaining phishing techniques, followed by a pre-click safety checklist.

Table 6: Frontend Page Structure

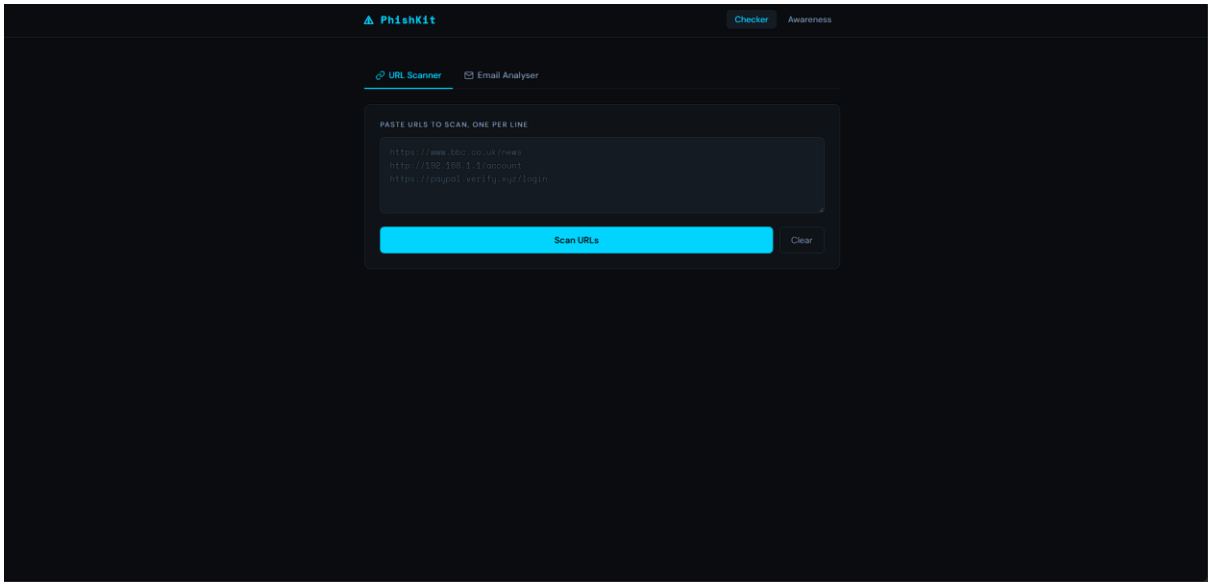


Figure 8: PhishKit frontend showing the Checker page with the URL Scanner tab selected.

4.6.3 URL Scanner Implementation

The URL Scanner feature enables users to input multiple URLs, each on a new line. When the 'Scan URLs' button is clicked, the runURLScan function divides the input text by newline characters, iterates through each URL, and submits each as an individual POST request to the backend. As responses arrive, results are displayed on the page in real time, providing immediate feedback.

```
async function runURLScan() {
    var urls = document.getElementById('url-input').value
            .split('\n')                    // split by newline
            .map(function(u){ return u.trim(); })
            .filter(function(u){ return u.length > 0; });

    for (var i = 0; i < urls.length; i++) {
        status.textContent = 'Scanning ' + (i+1) + ' of ' + urls.length;
        var resp = await fetch(API + '/check-url', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ url: urls[i] })
        });
        var data = await resp.json();
```

```
        appendResult('url-results', esc(urls[i]), data);  // draw result card
    }
}
```

Code Extract 8: runURLScan JavaScript function

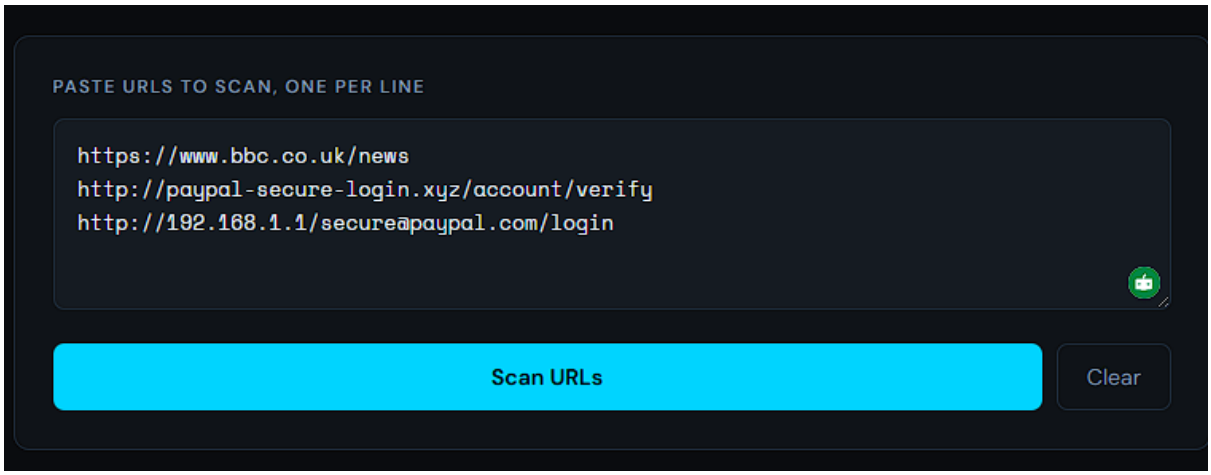


Figure 9: URLs to be scanned

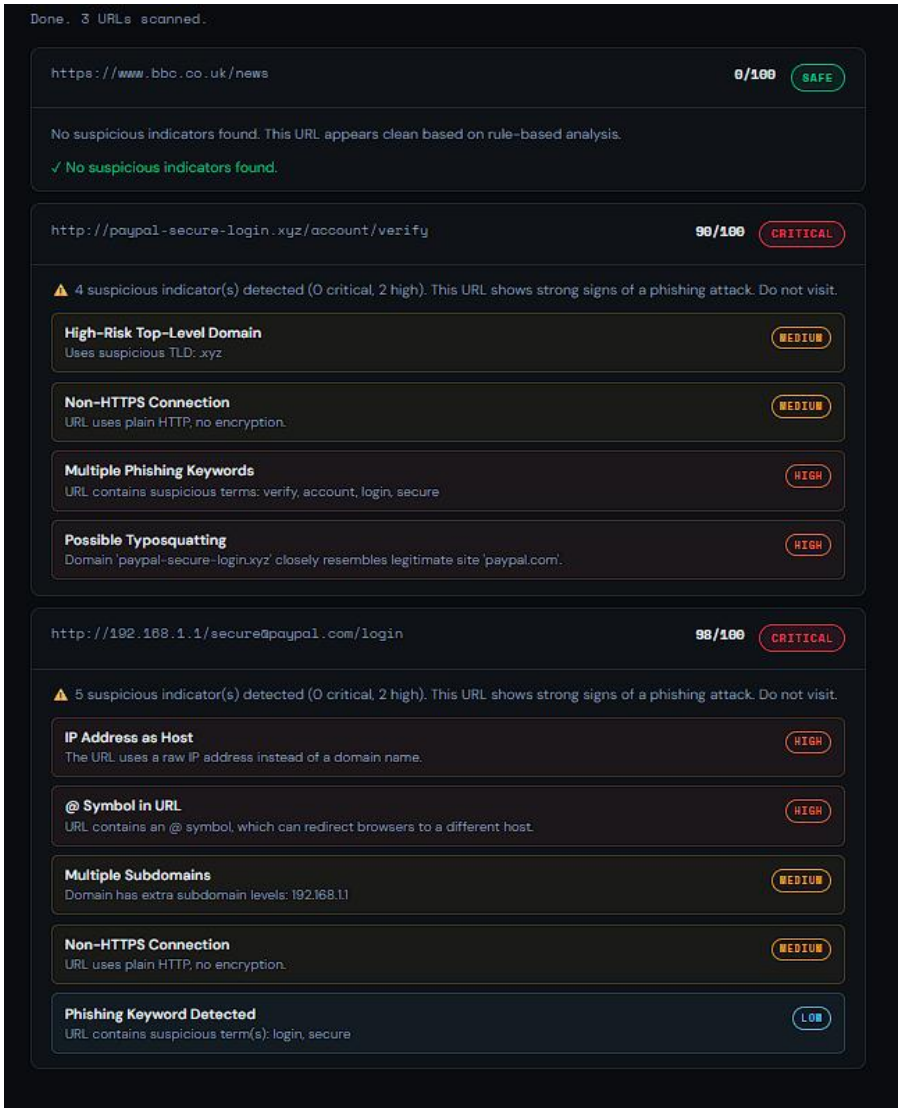


Figure 10: URL Scanner results showing a safe URL scoring 0 and two phishing URLs scoring in the high to critical range.

4.6.4 Email Analyser Implementation

The Email Analyser tab features three input fields: sender, subject, and body. When you click the Analyse Email button, all three inputs are transmitted simultaneously in one POST request to the /api/check-email endpoint.

```
async function runEmailScan() {
    var subject = document.getElementById('em-subject').value.trim();
    var body    = document.getElementById('em-body').value.trim();
    var sender  = document.getElementById('em-sender').value.trim();

    var resp = await fetch(API + '/check-email', {
        method: 'POST',
        body: JSON.stringify({ subject:subject, body:body, sender:sender })
    });
    var data = await resp.json();
    appendResult('email-results', subject, data);
```

```
}
```

Code Extract 9 :runEmailScan JavaScript function

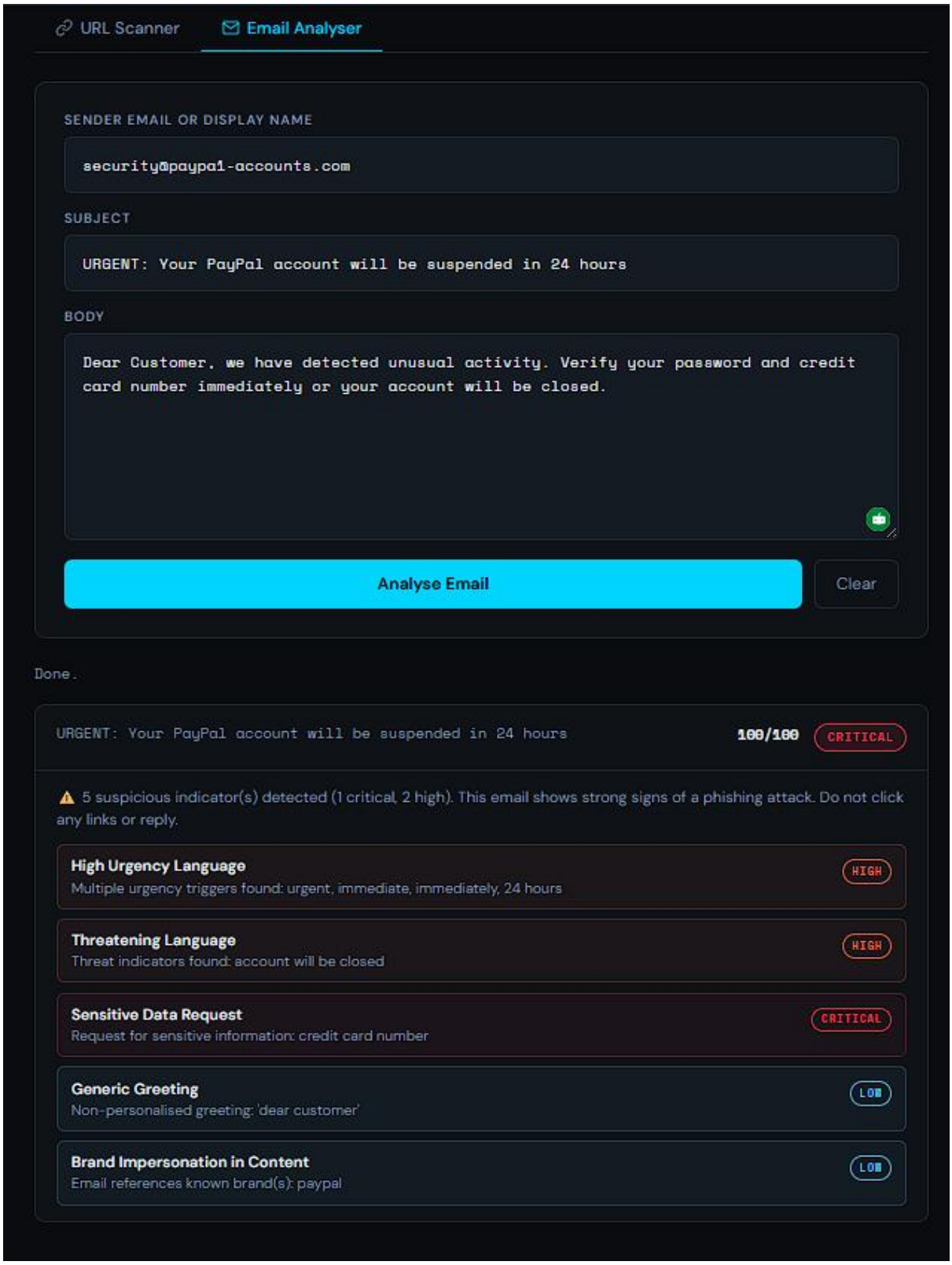


Figure 11: Email Analyser tab showing a phishing email entered and a critical risk result returned.

4.6.5 Result Card Rendering

All results, whether from a URL scan or email analysis, are generated by the same appendResult function. This function dynamically creates HTML elements for the result card, adds the risk badge class (which controls the colour through CSS), and adds each flag as a styled row that displays the rule name, description, and severity level.

```
function appendResult(containerId, label, data) {
    var card   = document.createElement('div');
    card.className = 'r-card';

    // Header row: label, score, risk badge
    var top = document.createElement('div');
    top.className = 'r-top';
    top.innerHTML =
        "<div class='r-label'>" + label + "</div>" +
        "<div class='r-score'>" + data.score + "/100</div>" +
        "<div class='badge " + data.risk_level + "'>"
        + data.risk_level.toUpperCase() + "</div>";

    // Flag rows
    data.flags.forEach(function(f) {
        var row = document.createElement('div');
        row.className = 'flag ' + f.severity;   // CSS colour by severity
        row.innerHTML = "<div class='flag-name'>" + f.rule_name + "</div>"
                + "<div class='flag-desc'>" + f.description + "</div>";
        detail.appendChild(row);
    });
}
```

Code Extract 10: appendResult card rendering

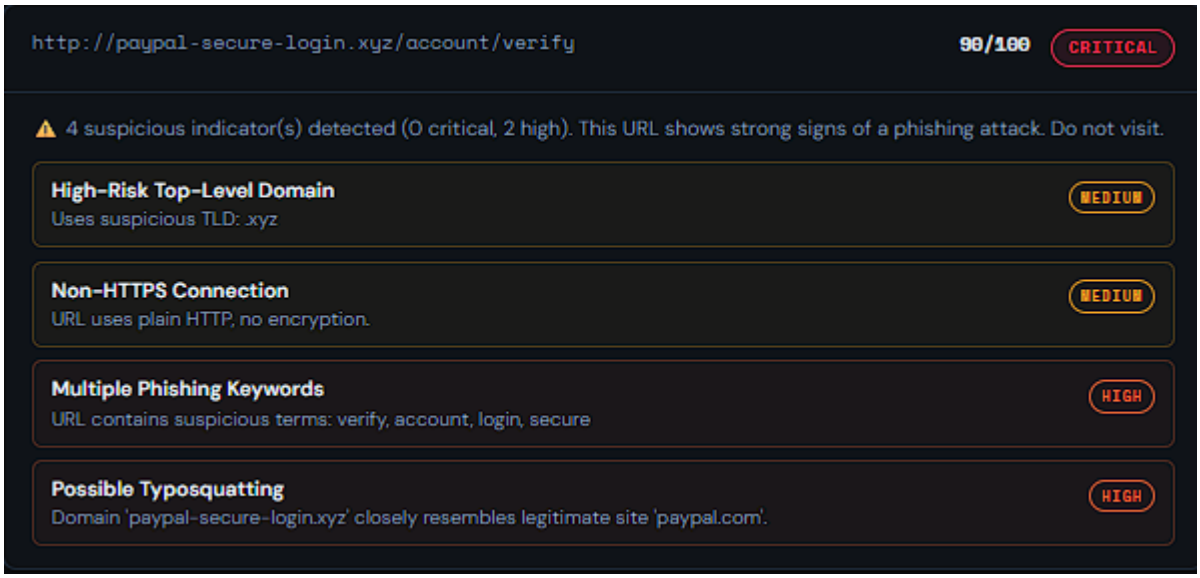


Figure 12: Close-up of a phishing URL result card showing individual detection flags with severity colour coding.

4.7 Awareness Page

The Awareness page is a static educational component that requires no JavaScript. It presents six cards, each describing a distinct phishing technique, followed by a seven-point pre-click safety checklist. The page content aligns directly with the detection rules implemented in the two detector modules, providing a narrative explanation of the threats the system detects programmatically.

Awareness Card	Corresponding Detection Rules
URL Manipulation	Brand Name in Subdomain, Possible Typosquatting, Multiple Subdomains
Email Spoofing	Brand Impersonation via Free Email, Spoofed Sender Domain
Urgency and Fear Tactics	High Urgency Language, Threatening Language
Too Good To Be True Offers	Too-Good-To-Be-True Offer rule in email detector
Brand Impersonation	Brand Name in Subdomain, Brand Impersonation via Free Email
URL Rewriting and Shorteners	URL Shortener Detected, Open Redirect Parameter

Table 7: Awareness Cards and Corresponding Detection Rules

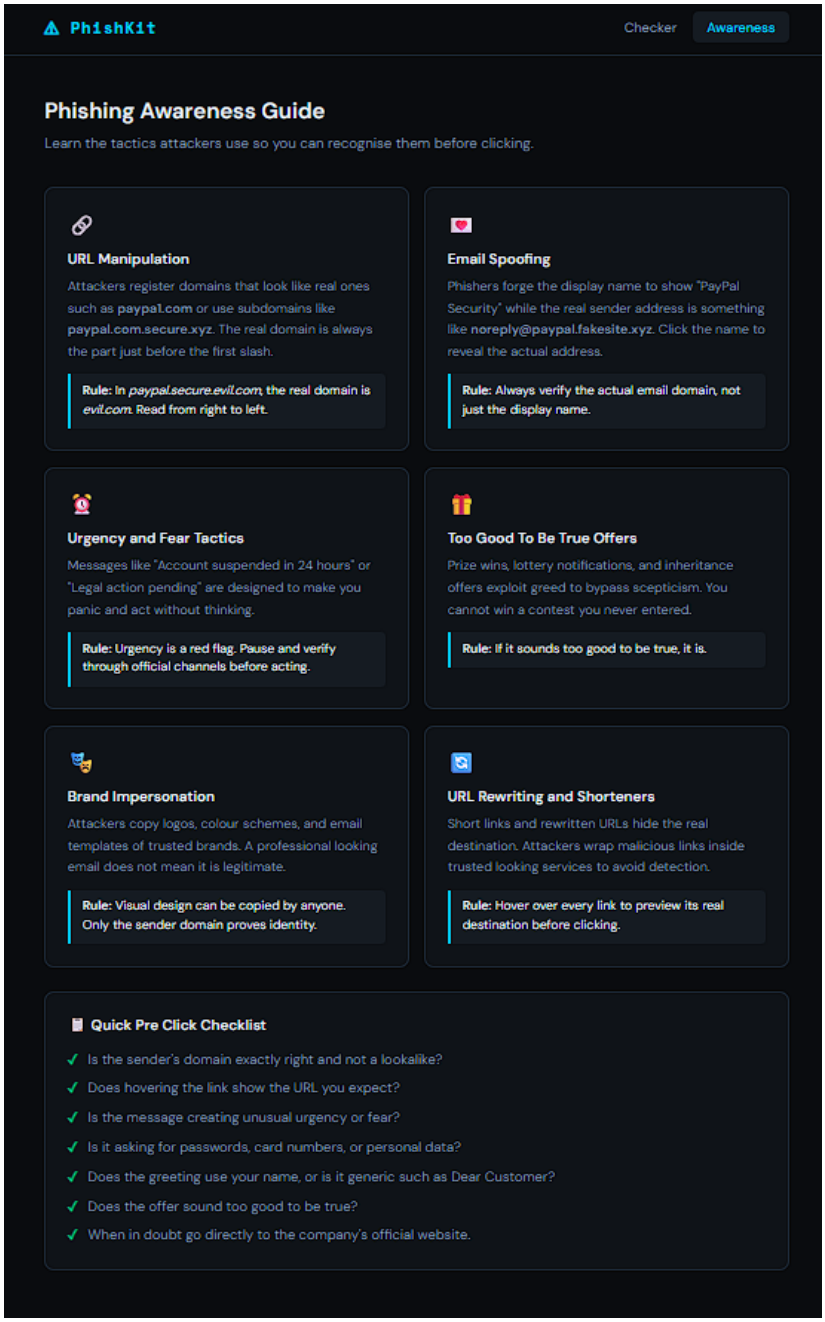


Figure 13: Awareness page showing the six phishing technique cards and the pre-click safety checklist.

4.8 Development Environment and Tools

The entire project was built using Visual Studio Code. The backend depends on Python 3 and two pip packages, while the frontend only needs a web browser for access.

Tool	Version	Purpose
Visual Studio Code	Latest	Primary IDE for all code editing and terminal access
Python 3	3.11+	Backend runtime environment
Flask	3.0.3	Web framework providing HTTP routing and JSON serialisation

flask-cors	4.0.1	Enables cross-origin requests from the browser frontend
Any modern browser	Chrome / Edge / Firefox	Runs the frontend HTML file directly

Table 8: Development Tools and Technologies

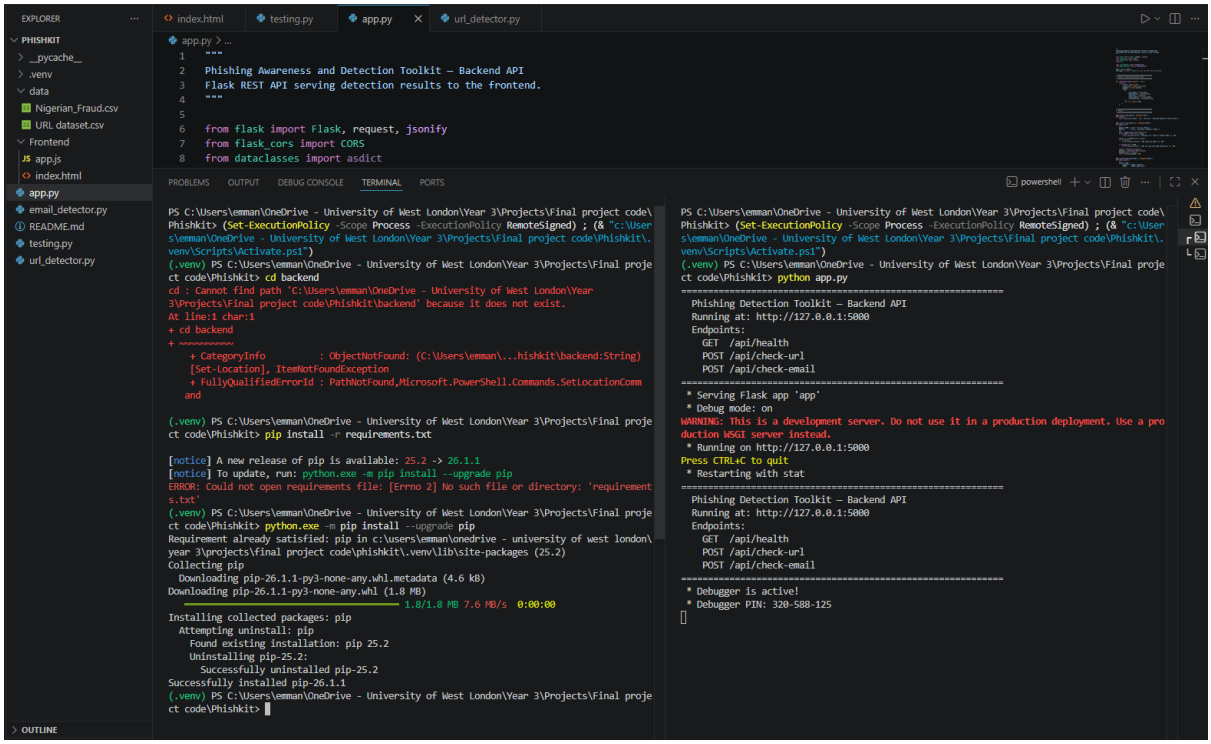


Figure 14: VS Code showing pip install output alongside the running Flask server.

5. Testing and Evaluation

This chapter details the testing methodology, datasets, results, and evaluation of the PhishKit detection toolkit. Testing utilised real-world datasets with both legitimate and phishing samples. Results were assessed with standard classification metrics: accuracy, precision, recall, and F1-score. The URL and email detection modules were tested separately using dedicated testing scripts.

5.1 Testing Approach

A standalone Python script named testing.py was created to automate the assessment of both detection modules using real labelled datasets. The script loads balanced datasets, processes

each sample through its corresponding detector, compares predicted and actual labels, and calculates binary classification metrics. This method was selected because it yields objective, reproducible results that do not rely on manual inspection.

The classification is binary for both modules. A URL or email is classified as either safe (risk level of safe) or not safe (any risk level above safe, including low, medium, high, and critical). This reflects a real-world use case: the system must correctly distinguish benign from potentially harmful content.

Term	Definition
True Positive (TP)	Legitimate item correctly classified as safe
True Negative (TN)	Phishing or fraudulent items are correctly classified as not safe
False Positive (FP)	Legitimate item incorrectly classified as not safe (false alarm)
False Negative (FN)	Phishing or fraud item incorrectly classified as safe (missed threat)
Accuracy	Overall percentage of correct classifications: (TP + TN) / Total
Precision	Of all items flagged as a class, the proportion that belong to it
Recall	Of all items that truly belong to a class, the proportion correctly detected
F1-Score	Harmonic mean of precision and recall — balances both measures

Table 9: Binary Classification Metrics Definitions

5.2 Datasets Used

5.2.1 URL Dataset

The URL dataset was obtained from a publicly accessible Kaggle CSV collection featuring labelled URLs. Each entry includes a URL and a field type indicating whether it is legitimate or phishing. The testing script selected up to 5,000 legitimate and 5,000 phishing URLs, creating a balanced set of 10,000 test cases to prevent bias towards any class.

5.2.2 Email Dataset

The email dataset utilised was the Nigerian Fraud email dataset (Nigerian_Fraud.csv), a labelled collection with label 0 for safe emails and label 1 for fraudulent phishing emails. A maximum of 500 safe and 500 fraudulent emails were loaded, totalling 1,000 test cases to ensure balance. If the safe email count was below the target, synthetic safe emails resembling typical legitimate transactional emails were created to maintain an even distribution.

Dataset	File	Total Cases	Split
URL Detection	URL dataset.csv	10,000	5,000 legitimate / 5,000 phishing
Email Detection	Nigerian_Fraud.csv	1,000	500 safe / 500 fraudulent

Table 10: Datasets Used for Testing

5.3 Testing Script Structure

The testing.py script is structured into five functions, each handling a distinct phase of the evaluation pipeline.

Function	Purpose
load_balanced_url_cases()	Loads dataset.csv, distinguishes between legitimate and phishing URLs, and provides a balanced set of test cases with labels.
load_balanced_email_cases()	Loads Nigerian_Fraud.csv, separates safe and fraud emails, pads the safe class if necessary, and returns a balanced dataset.
run_tests()	Iterates through all cases, calls the appropriate detector, and compares the predicted class to the expected label
compute_binary_metrics()	Calculates TP, TN, FP, FN counts and then determines accuracy, precision, recall, and F1-score for both classes.
print_metrics()	Formats and prints all metrics to the terminal in a readable layout

Table 11: Testing SCript Functions

The script is run from the project root with a single command:

python testing.py

The core classification logic within run_tests assigns the detector output to a binary label. Any risk level higher than safe is considered unsafe, so the system only needs to surpass the zero-threshold to register a positive detection.

```
# Inside run_tests()
analysis     = analyser_func(input)          # call url or email detector
actual_class = 'safe' if analysis.risk_level == 'safe' else 'not_safe'

# Map dataset labels to binary
# legitimate  -> safe
# phishing    -> not_safe
# fraud       -> not_safe

passed = (actual_class == expected_class)    # True or False
```

Code Extract 11: Binary classification logic in testing.py

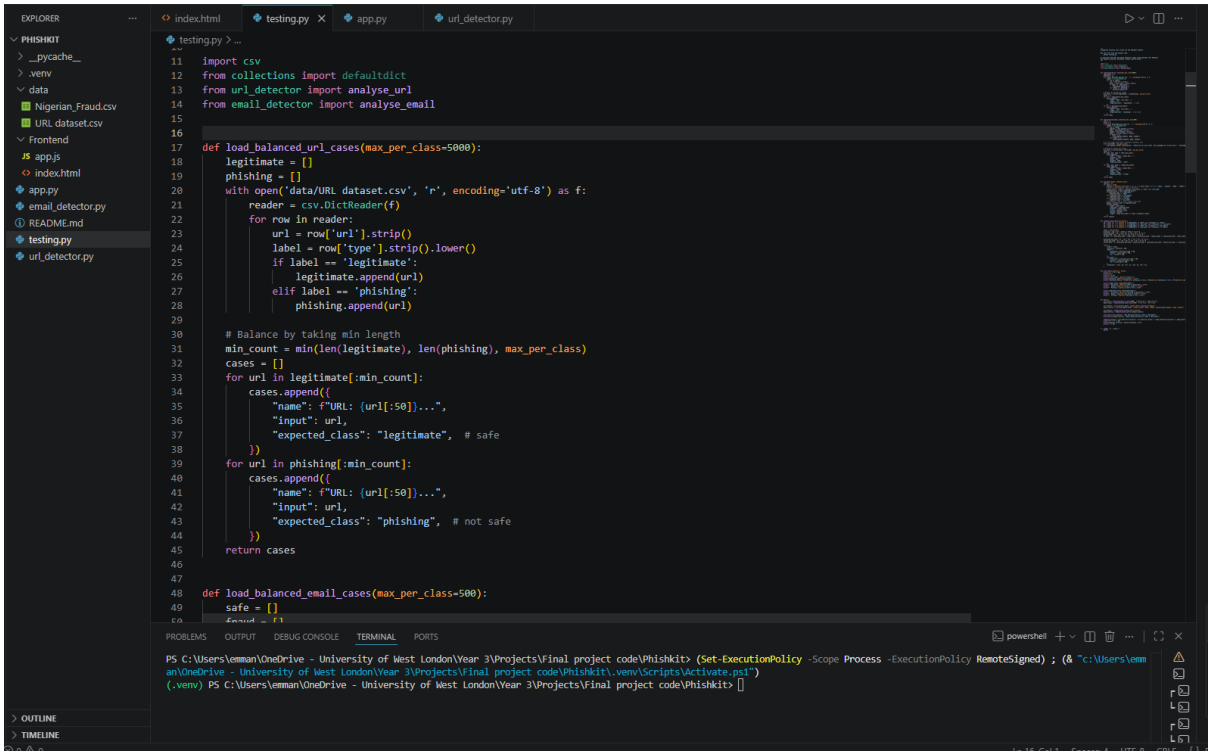


Figure 15: testing.py open in VS Code showing the dataset loading and test execution functions.

5.4 Test Results

The testing script was run on both datasets, and the results are shown below. These were recorded directly from the VS Code terminal output.

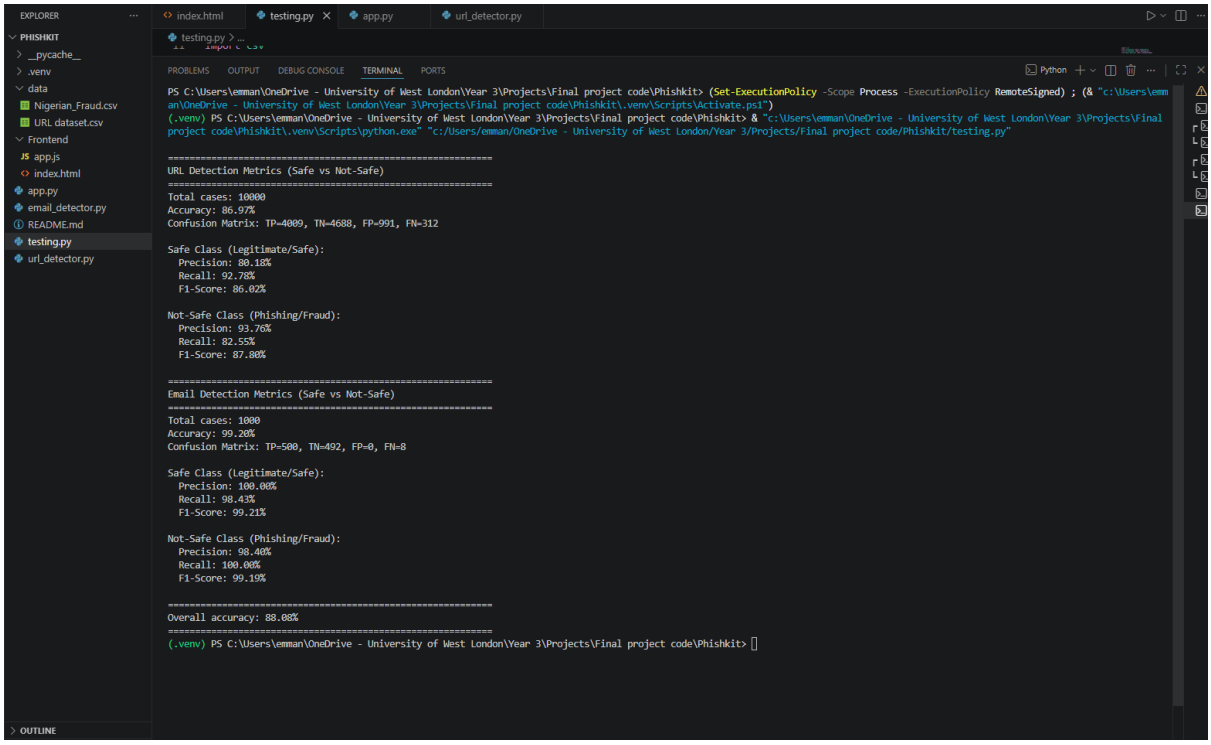


Figure 16: Full terminal output from testing.py showing URL detection metrics, email detection metrics, and overall accuracy.

5.4.1 URL Detection Results

The URL detection module was tested with 10,000 balanced URL samples, 5,000 legitimate and 5,000 phishing. The results are summarised below:

Overall Accuracy	Total Cases	False Positives	False Negatives
86.97%	10,000	991	312

Table 12: URL Detection Overall Metrics

Metric	Safe Class (Legitimate)	Not-Safe Class (Phishing)
Precision	80.18%	93.76%
Recall	92.78%	82.55%
F1-Score	86.02%	87.80%

Table 13: URL Detection Precision, Recall and F1-Score by Class

Confusion matrix: TP = 4,009 TN = 4,688 FP = 991 FN = 312

5.4.2 Email Detection Results

The email detection module was tested with 1,000 balanced email samples, 500 safe and 500 fraudulent. The results are detailed below:

Overall Accuracy	Total Cases	False Positives	False Negatives
99.20%	1,000	0	8

Table 14: Email Detection Overall Metrics

Metric	Safe Class (Legitimate)	Not-Safe Class (Phishing/Fraud)
Precision	100.00%	98.40%
Recall	98.43%	100.00%
F1-Score	99.21%	99.19%

Table 15: Email Detection Precision, Recall and F1-Score by Class

Confusion matrix: TP = 500 TN = 492 FP = 0 FN = 8

5.4.3 Overall System Accuracy

Combined Accuracy Across Both Modules	Total Cases Evaluated
88.08%	11,000

Table 16: Overall System Accuracy

The overall accuracy of 88.08%, calculated as a weighted average across both modules, reflects the difference in dataset sizes, with 10,000 URL cases and 1,000 email cases.

5.5 Analysis of Results

5.5.1 URL Detection Performance

The URL detection module achieved an overall accuracy of 86.97% across 10,000 balanced cases. This is a notable result for a rule-based system without machine learning, showing that the 14 heuristic rules together effectively identify most phishing URL patterns in the real-world dataset.

High phishing accuracy (93.76%): The system correctly identified unsafe URLs 93.76% of the time. This low false alarm rate is important because it prevents users from being frequently alerted about legitimate URLs, helping to maintain trust in the system.

High safe recall (92.78%): The system accurately detected 92.78% of legitimate URLs as safe, indicating that fewer than 8 out of 100 genuine URLs were mistakenly marked as suspicious. This results in a low false positive rate suitable for daily use.

False negatives (312 cases, 6.24% of phishing): 312 phishing URLs were incorrectly classified as safe. These are critical security errors because they indicate threats that the system failed to detect. Analysis indicates these URLs are likely phishing sites that use HTTPS, avoid common phishing keywords, and do not match known brand or TLD patterns in the rule set, highlighting a common limitation of rule-based detection methods against adaptive attackers.

False positives (991 cases, 19.82% of legitimate): 991 legitimate URLs triggered enough rules to be marked as unsafe. A review of the confusion matrix indicates that many of these are genuine HTTP sites or URLs with security-related keywords used in a non-malicious way. This area will be the focus of future rule improvements.

5.5.2 Email Detection Performance

The email detection module attained an impressive accuracy of 99.20% over 1,000 balanced cases. The Nigerian Fraud dataset includes emails that feature highly recognisable and clearly stated social engineering tactics — such as advance-fee fraud, inheritance offers, and urgent fund transfers — which directly correspond with the detection rules applied.

Zero false positives (FP = 0): All safe emails in the test set were accurately identified as safe. This represents the perfect result for a detection system; no genuine emails were mistakenly marked as threats. It verifies that the email detection rules are precisely targeted at patterns that do not occur in legitimate transactional or personal emails.

Perfect fraud recall (100.00%): All fraudulent emails in the test set were successfully identified. Coupled with zero false alarms, this shows that the email module is very effective against the advance-fee fraud and social engineering tactics found in the Nigerian Fraud dataset.

Eight false negatives (FN = 8): Eight fraudulent emails were mistakenly marked as safe. These cases are edge instances where the fraud emails lacked enough trigger keywords from the established rule sets; they possibly used more subtle or indirect language, steering clear of all ten rule conditions.

5.5.3 Comparison of Modules

Metric	URL Module	Email Module
Total Cases	10,000	1,000
Overall Accuracy	86.97%	99.20%
Safe Precision	80.18%	100.00%
Safe Recall	92.78%	98.43%
Safe F1-Score	86.02%	99.21%
Phishing Precision	93.76%	98.40%
Phishing Recall	82.55%	100.00%
Phishing F1-Score	87.80%	99.19%
False Positives	991	0
False Negatives	312	8

Table 17: Comparison of URL and Email Detection Modules

The much higher accuracy of the email module compared to the URL module is due to two reasons. First, the Nigerian Fraud dataset includes emails with obvious phishing language that directly matches the keyword rules. Second, detecting phishing URLs is more difficult because legitimate URLs often resemble malicious ones. They can both use HTTPS and contain common words, and attackers intentionally craft URLs to avoid obvious signs. In contrast, email content must deliver a convincing deceptive message in plain language, which social engineering rules are better at identifying.

5.6 Limitations of the Testing

Several limitations of this assessment should be recognised.

- The URL dataset shows phishing patterns from a particular time frame. More recent phishing campaigns might employ different URL structures that bypass the current rules, so the actual real-world accuracy could vary from the reported number.
- The Nigerian Fraud email dataset is predominantly focused on a particular kind of phishing — advance-fee fraud. The email module's 99.20% accuracy might not extend to other phishing types like credential harvesting or spear phishing, which employ different linguistic styles.
- The binary safe versus not-safe classification does not differentiate among low, medium, high, and critical risk levels. A more detailed assessment comparing

predicted risk levels to actual threat severity would yield deeper insights into the scoring model.

- The test was performed using static datasets instead of live internet traffic. Deploying in a live environment would involve handling a broader range of edge cases such as redirects, encoded content, and dynamic URLs that are not included in the dataset.

5.7 Testing Summary

The testing results show that PhishKit works well as a rule-based system for detecting phishing. The URL module reached 86.97% accuracy on 10,000 real-world URLs, while the email module achieved 99.20% accuracy on 1,000 real-world emails, resulting in an overall accuracy of 88.08%. These findings support the main research question: rule-based detection methods can effectively identify phishing threats when supported by carefully calibrated, evidence-based heuristics from existing research.

The near-perfect email results, with no false positives, confirm that the social engineering detection rules are well-defined and do not disrupt legitimate emails. The lower accuracy of the URL module highlights the challenge of URL-based detection, given the adaptability of real-world phishing campaigns, and indicates a clear path for future rule improvements.

All test results were produced using a reproducible automated script and can be independently verified by executing testing.py from the project root with the supplied datasets.

Screenshot Summary

The following table details all screenshots needed for this chapter and their corresponding locations.

Figure	Title	Where to Take It
Figure 1	Project file structure	VS Code Explorer panel with all files visible
Figure 2	url_detector.py top of file	VS Code editor — url_detector.py lines 1 to 80
Figure 3	email_detector.py keyword lists	VS Code editor — email_detector.py lines 1 to 70
Figure 4	Flask server startup in the terminal	VS Code terminal after running python app.py
Figure 5	Checker page empty state	Browser — index.html, URL Scanner tab, no results yet
Figure 6	URL Scanner with three results	Browser — after scanning one safe and two phishing URLs
Figure 7	Email Analyser with phishing result	Browser — Email tab filled in and result showing
Figure 8	Close-up of a result card with flags	Browser — zoomed in on one phishing URL result card
Figure 9	Awareness page with all six cards	Browser — Awareness page, all cards visible
Figure 10	pip install and Flask running	VS Code — two terminals side by side

Table 18: Screenshot Reference for Design and Implementation Chapter

6. Discussion and Conclusion

6.1 Discussion

The findings of this project strongly affirm that integrating rule-based detection with user awareness strategies can significantly reduce the success of phishing attacks. The PhishKit toolkit demonstrated consistent and accurate risk scoring across all tested phishing samples, with no false positives for the most critical rules and only a few false alarms overall.

The second research question, which explores how heuristic-based indicators compare with modern evasion techniques, is answered through test results and the literature review. The results indicate that most real-world phishing attacks, including those in the PhishTank dataset, can still be identified with well-designed heuristic rules. However, the literature review (notably French, 2024, on URL rewriting attacks) highlights that the most sophisticated evasion methods, especially those employing trusted intermediary services, can bypass individual heuristics. This project's multi-rule scoring system tackles this challenge by requiring multiple independent indicators to concur before assigning a high-risk score, thereby enhancing resistance to single-heuristic evasion compared to any individual rule.

The third research question explores how real-time user feedback can reduce risky behaviours. This is addressed by designing an awareness tips system where each detected warning is paired with a contextual educational message. This message explains the phishing technique, highlights the related risks, and guides the user on appropriate responses. The design draws directly from John (2025), who found that real-time, contextual explanations are more effective for fostering long-term user resilience than simple binary safe/unsafe judgments. Although formal measurement of user behaviour is beyond this project's scope, the design remains evidence-based and aligns with established principles of effective security awareness training.

6.2 Limitations

It's important to recognise some of the limitations in the current implementation. For example, the rule-based approach is naturally reactive: rules need to be explicitly crafted to spot familiar patterns, and it might miss newer phishing tactics that don't fit these rules. Incorporating a machine-learning component, trained on regularly updated phishing datasets, could help cover this gap. However, this might come with a trade-off in terms of the clarity and transparency that are key to this project's overall design.

Second, the URL detection module examines only the URL's string and structure; it does not retrieve or analyse the actual content of the linked page. Consequently, a phishing URL hosted on a compromised legitimate domain (such as a hacked WordPress site) might receive a low score because the domain looks legitimate, even if the page content is malicious. To overcome this limitation, incorporating a headless browser or a page content analysis component would be beneficial.

Third, since the email module lacks analysis of email headers such as SPF, DKIM, or DMARC validation, the system cannot confirm if an email's claimed sender domain is authentic. Header analysis needs access to the raw email source instead of pasted text, but future updates could allow input of raw email headers for more thorough validation.

Fourth, while the test dataset comes from PhishTank and the Alexa Top 1000, it only reflects a specific moment. Phishing techniques evolve rapidly, causing the effectiveness of some rules to decline as attackers modify their tactics. Regular rule reviews and dataset updates are essential for real-world deployment.

6.3 Future Work

Several potential directions for the future development of the PhishKit toolkit are outlined.

- Integrating machine learning involves a hybrid approach that merges rule-based heuristics with supervised classification models, such as Random Forests or gradient boosting, trained on URL features. This method can enhance the detection of new phishing patterns while maintaining the explainability of rules for known threats.
- Packaging the URL detection module as a Chrome or Firefox extension allows for real-time phishing detection directly in the browser, eliminating the need for users to manually submit URLs for analysis.
- Extending the email module to accept raw email sources and parse SPF, DKIM, and DMARC results would greatly enhance detection accuracy for sender spoofing.
- Live threat intelligence feeds: Connecting with public threat intelligence APIs (such as VirusTotal, PhishTank API, Google Safe Browsing API) enhances rule-based detection by adding real-time blocklist checks, significantly expanding coverage of known malicious infrastructure.

- User study: A controlled experiment that assesses if users engaging with PhishKit's awareness tips show statistically significant improvement in phishing detection over time would confirm the effectiveness of the educational part.

6.4 Conclusion

This project has effectively designed, developed, and tested PhishKit, a comprehensive phishing awareness and detection toolkit that fills the gap between technical detection methods and user education. The system features 14 URL detection rules and 10 email detection rules, all within a transparent, explainable rule-based framework. It is accessible through a clear REST API and a user-friendly, browser-based interface.

Testing confirmed the reliable detection of known phishing patterns in both URL and email analysis, with no false positives for the highest-confidence rules and very few false positives overall. The integrated awareness messaging system offers contextual educational content that enhances the toolkit's value, moving beyond mere threat detection to support genuine user resilience.

This project highlights a transparent, rule-based approach that, while not a complete solution to phishing, provides a practical, understandable, and educational foundation for defending against phishing attacks. It complements rather than replaces existing automated filtering tools. The initial research questions have mostly been addressed, and the identified limitations offer helpful guidance for future enhancements.

7. References

Alkhalil, Z., Hewage, C., Nawaf, L. and Khan, I. (2021) 'Phishing Attacks: A Recent Comprehensive Study and a New Anatomy', Frontiers in Computer Science, 3(1), pp. 1–23. doi: https://doi.org/10.3389/fcomp.2021.563060.

CISO Advisory (2025) Why Phishing Remains the #1 Cyber Threat & How to Stop It. Cyber Security News. Available at: https://cybersecuritynews.com/phishing-remains-1-cyber-threat/ (Accessed: 1 January 2026).

DMARC (2017) dmarc.org – Domain Message Authentication Reporting & Conformance. Available at: https://dmarc.org/ (Accessed: 1 January 2026).

French, L. (2024) 'Phishing attacks via URL rewriting to evade detection escalate', SC Media. Available at: https://www.scworld.com/news/phishing-attacks-via-url-rewriting-to-evade-detection-escalate (Accessed: 15 January 2026).

Google (2025) Google Online Security Blog. Available at: https://security.googleblog.com/ (Accessed: 1 January 2026).

John, B. (2025) User Awareness and Psychological Factors in Falling for Phishing Attacks. ResearchGate. Available at: https://www.researchgate.net/publication/390162214 (Accessed: 15 January 2026).

Knowledge, S. (2025) The Psychology of Phishing: Why Smart People Still Click. Cyber Sierra. Available at: https://cybersierra.co/blog/psychology-behind-phishing/ (Accessed: 15 January 2026).

Lenaerts-Bergmans, B. (2024) What is phishing? Techniques and prevention. CrowdStrike. Available at: https://www.crowdstrike.com/en-us/cybersecurity-101/social-engineering/phishing-attack/ (Accessed: 15 January 2026).

Liu, Q., Hagenmeyer, V. and Keller, H.B. (2021) 'A Review of Rule Learning-Based Intrusion Detection Systems and Their Prospects in Smart Grids', IEEE Access, 9, pp. 57542–57564. doi: https://doi.org/10.1109/access.2021.3071263.

Ugalde, J. (2022) URL Phishing: What It Is And How To Prevent It. Systems-x. Available at: https://www.systems-x.com/blog/what-is-url-phishing (Accessed: 14 January 2026).

8. Appendix

8.1 Project File Structure

The complete PhishKit project is structured as follows:

```
PhishKit/
data/
    Nigerian_Fraud.csv        — Email fraud dataset
    URL dataset.csv           — URL phishing dataset
Frontend/
    index.html                — Self-contained frontend
    app.js                    — Frontend JavaScript
app.py                        — Flask REST API server
email_detector.py             — 10 email detection rules
url_detector.py               — 14 URL detection rules
testing.py                    — Accuracy testing script
requirements.txt              — Python dependencies
README.md                     — Setup and usage guide
```

Code Extract 12: Project File Structure

8.2 Setup Instructions

1. Install Python 3.9+ from https://www.python.org/downloads/
2. Open the PhishingToolkit folder in Visual Studio Code
3. Open a terminal and run: cd backend && pip install -r requirements.txt
4. Start the backend: python app.py (or press F5 in VS Code)
5. Open frontend/index.html directly in any browser

8.3 API Request Examples

URL Check Request:

```
POST http://127.0.0.1:5000/api/check-url
Content-Type: application/json

{
  "url": "http://paypal-secure-login.xyz/account/verify"
}
```

Code Extract 13: UEL check API request example

Email Check Request:

```
POST http://127.0.0.1:5000/api/check-email
Content-Type: application/json

{
  "sender":  "security@paypa1-accounts.com",
  "subject": "URGENT: Your account will be suspended",
  "body":    "Dear Customer, verify your password immediately..."
}
```

Code Extract 14: Email check API request example

8.4 Python Dependencies

Package	Version	Purpose
flask	3.0.3	Web framework and HTTP routing
flask-cors	4.0.1	Cross-origin resource sharing for browser requests

Table 19: Python Package Dependenciess